

Technical Architecture Document

ShopAssist AI

E-commerce Customer Support Automation

Technical Overview

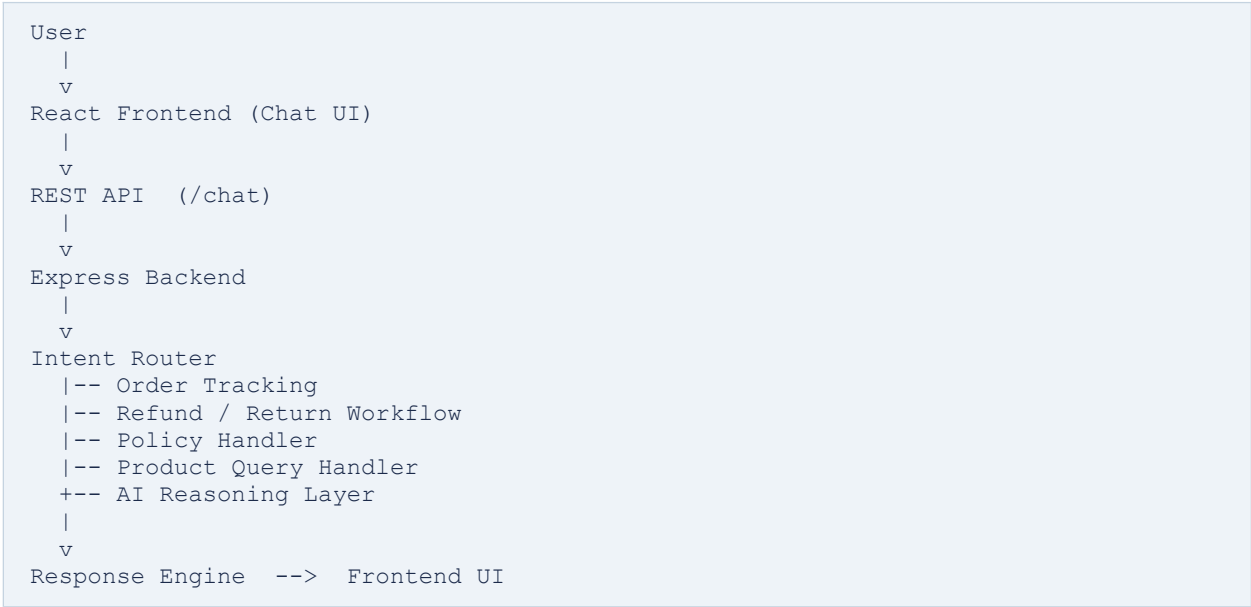
ShopAssist AI is a full-stack AI-powered customer support assistant built for e-commerce support automation. It combines deterministic workflow handling for operational reliability with LLM-based reasoning for natural language customer interactions.

The system automates the following support functions:

- Order tracking
- Return and refund initiation
- Policy support
- Product FAQs
- Human escalation
- Conversational AI assistance

1. System Architecture

ShopAssist AI follows a client-server hybrid architecture.



Design Approach

Structured workflows are handled deterministically for speed and reliability. **Natural language queries** that require reasoning are routed to the AI layer.

This hybrid approach reduces hallucination risk while preserving conversational flexibility.

2. Frontend

Stack

- React
- Vite
- CSS
- Fetch API

Responsibilities

The frontend renders the chat interface, collects user input, maintains chat state, displays responses, and communicates with backend APIs. A chat-based interface was chosen to make support interactions intuitive and low-friction.

3. Backend

Stack

- Node.js
- Express.js
- Axios
- dotenv
- CORS

Responsibilities

The backend acts as the orchestration layer for all support operations. It handles API request processing, intent detection, workflow execution, product data lookup, AI integration, and return request logging.

4. API Design

Health Check

```
GET /
```

Used to verify backend availability.

Chat Endpoint

```
POST /chat
```

Example request:

```
{
  "message": "Where is my order?"
}
```

Example response:

```
{  
  "reply": "Sure. Please provide your order ID to track your shipment."  
}
```

5. Workflow Engine

Order Tracking

1. Detect intent
2. Request order ID
3. Validate order
4. Return shipment status

Refund / Return

5. Detect refund intent
6. Request order ID
7. Validate order
8. Log return request
9. Confirm initiation

Policy Support

Handles queries related to return policy, refund policy, and shipping policy.

Product Support

Handles waterproof queries, warranty questions, size availability, and compatibility. Known product queries are handled directly for faster and more reliable responses.

6. AI Integration

Provider

OpenRouter API

Purpose

AI is used only when structured workflows are insufficient. Example queries routed to the AI layer:

- "Can I wear this watch while swimming?"
- "I changed my mind about my purchase."
- "I bought shoes but they don't fit."

The backend injects product and policy context into the prompt so responses remain grounded.

Reliability Strategy

Multiple fallback models are configured to reduce provider failure risk.

7. Data Layer

Mock structured storage was used to enable rapid prototyping without database overhead:

- orders.json
 - products.json
 - policies.json
 - returns.json
-

8. Deployment

Frontend

Hosted on Vercel.

Backend

Hosted on Render.

Source Control

GitHub. Environment variables are used for secure API key handling.

9. Key Technical Decisions

Hybrid AI Architecture

Chosen over a pure-AI approach to improve reliability and reduce hallucinations.

JSON Storage over Database

Chosen for speed of implementation within hackathon time constraints.

Deterministic Handling for Known Queries

Critical workflows such as refunds and order tracking are handled without AI dependency, ensuring fast and predictable responses.

Conclusion

ShopAssist AI was built as a practical, production-inspired support system rather than a generic chatbot. By combining workflow automation with contextual AI reasoning, the system delivers reliable operational support while maintaining natural conversational interaction.